Online Spreadsheet, a low latency html5 websocket Cloud service (SaaS) in C# .net (as Google Sheets)

Bharath K A

Build your own cloud spreadsheet (like Microsoft Excel, google sheets) that exposes shreadsheet workbook via web browser. Users can create excel like spreadsheets using web browser save it, share it, put formulas to compute data in the sheet etc.

From Wiki:

Reference (1)
"Google Docs, Sheets, Slides and Forms are a free, web-based word processor, a spreadsheet program, a presentation program and survey program respectively, all part of a software office suite offered by Google within its Google Drive service. The suite allows users to create and edit documents online while collaborating with other users in real-time"

Reference (2)
"Software as a service is a software licensing and delivery model in which software is licensed on a subscription basis and is centrally hosted. "

According to a Gartner Group estimate, SaaS sales in 2010 reached $10 billion, and were projected to increase to $12.1bn in 2011, up 20.7% from 2010. Gartner Group estimates that SaaS revenue will be more than double its 2010 numbers by 2015 and reach a projected $21.3bn"

Introduction

Many of us would have used Microsoft excel spreadsheets for work and personal needs for ages. Most of us are aware, for past few years, these worksheets are available online as a cloud service. Google sheets is one of many cloud computing document sharing services. Would you like to create an online cloud spreadsheet service (SaaS) for your customers or fun ? Read on..

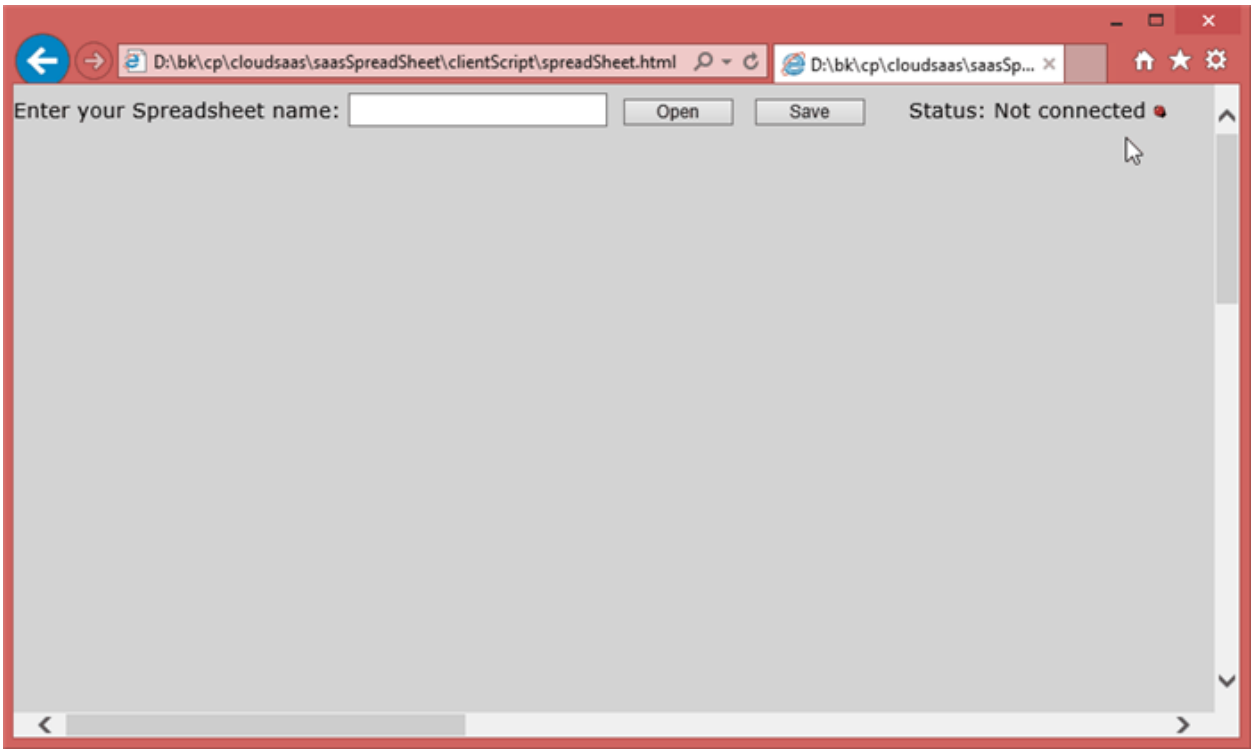


Fig1: Spread sheet open in Microsoft internet explorer. Note the computed (formula) cells are getting computed as we change the values in related cells

How would you like to proceed

- If you are looking for to check out this SaaS, goto Quick Demo section. You can setup this in few seconds and check it out right away
- If you would like to understand how it works, proceed reading on

Quick Demo

- Copy the demo binaries to your computer by clicking the link at top of this page
- Run saasSpreadSheet.exe. If you get firewall warning, click 'Allow' button to get the service started
- Open spreadSheet.html

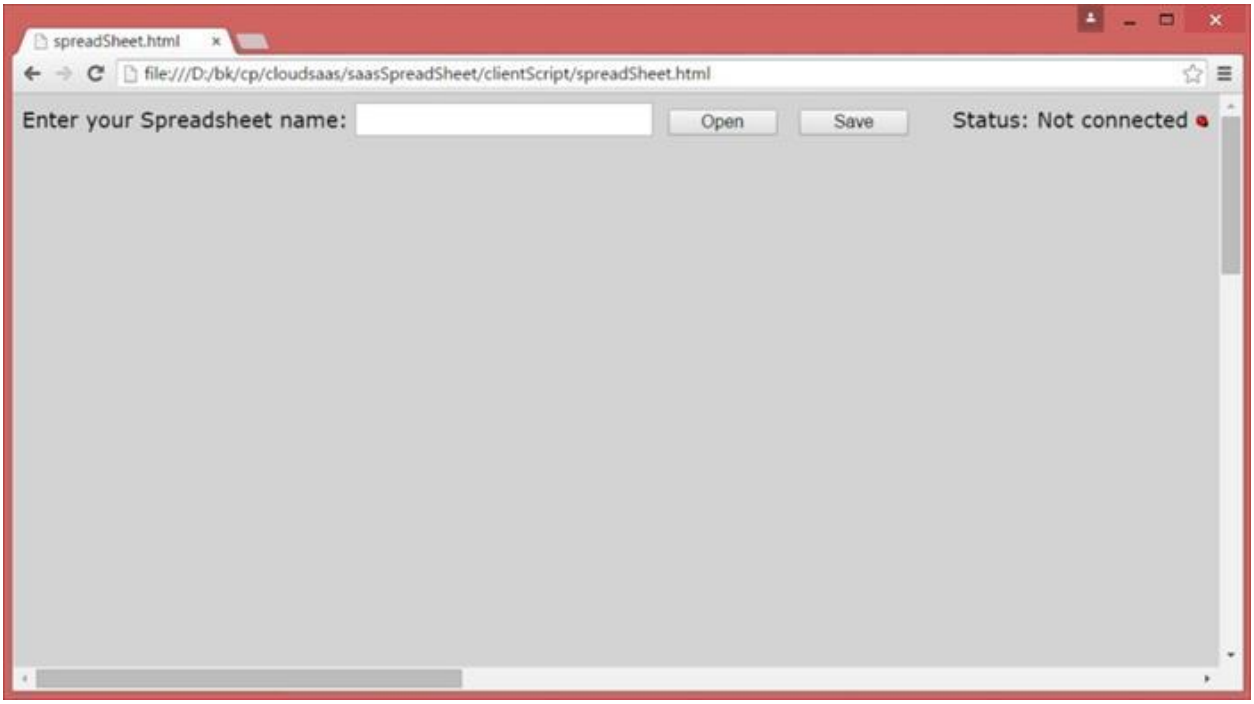


Fig 2: Spread sheet page not connected. Note, the blinking red light at right top. Browser used: google chrome

- I already have a sample sheet saved for you! So type in 'test' in the edit box as in below picture. Then, click Open button. You will see the spreadsheet loaded as in below picture if you have the service up and running. Modify the Quantity, Price or Long/ Short column. You will see the Position column computed automatically based on the formula. As you will know, you can save and retrieve the document.

In case if you would like to start fresh, key in a new name. Then, click Open. You will get a fresh sheet to start. Key in new data and math formula to play.

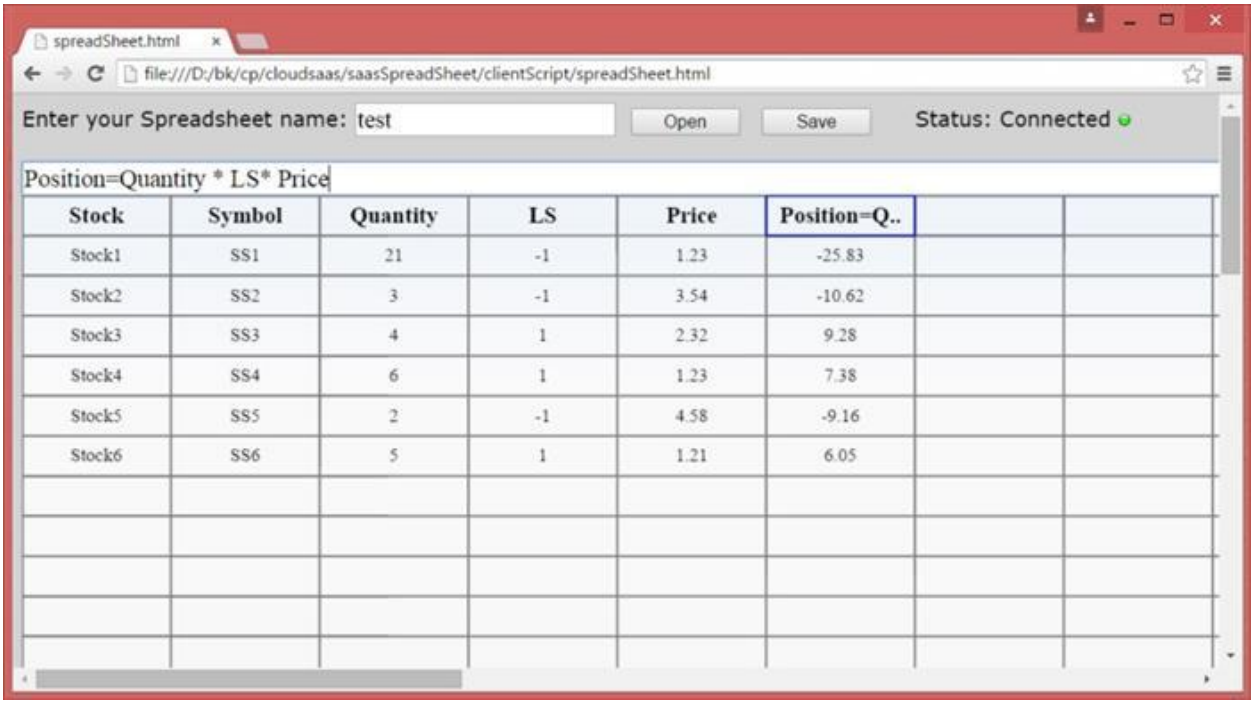


Fig3: Spreadsheet open and connected in google chrome. Note the green connected light at right top

- Kill the service. i.e., Close down the executable that you started in Step 2. You will see the status change to 'Not Connected'

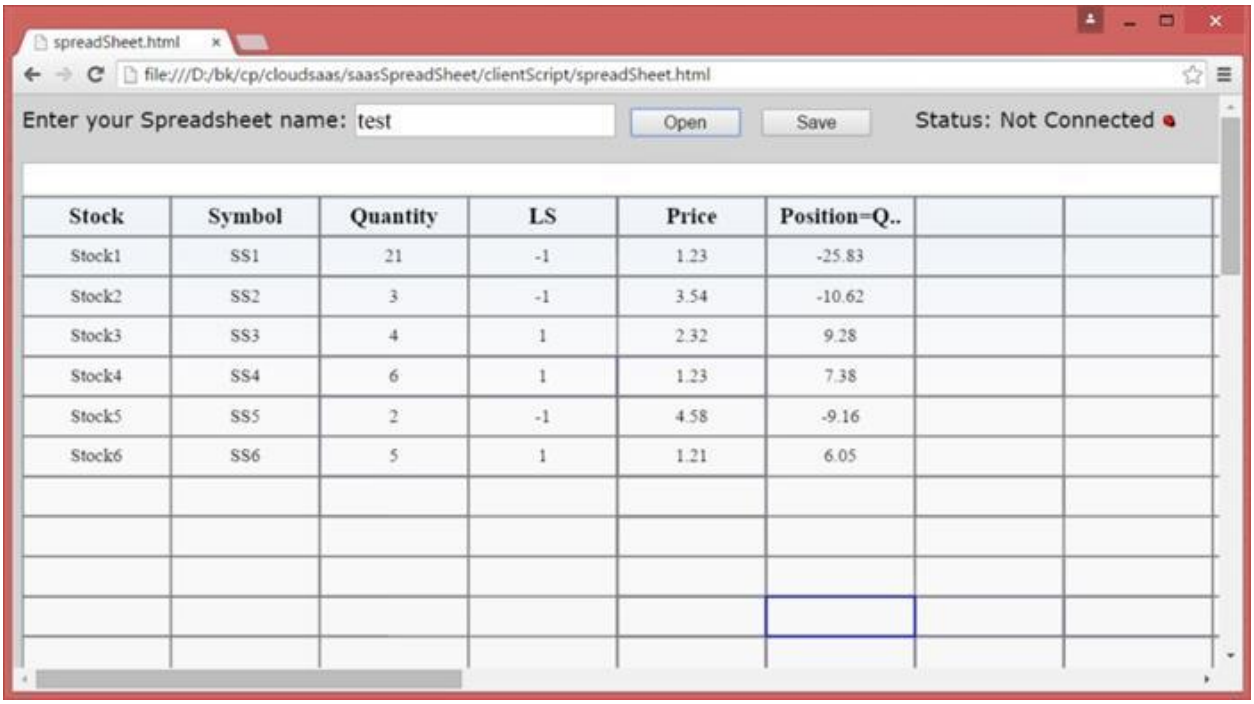


Fig 4: As the service is killed, the spreadsheet's blinking red light returns saying it's offline now. Browser used: google chrome

Why Cloud service

If you are already aware of advantages of cloud computing you can safely skip this section. If not, here is a brief details on advantages of to cloud platform
Recently, I sent an email with a file attachment. Unfortunately, next morning, they confirmed they *did not* receive it. Guess what, file size was off-limit, huge. Job delayed - time is money. Well, I could have used cloud service to send him that damn file. Everyone of us would have gone through this experience. This begins advantages of cloud services:

- Cost effective: Traditional desktop/enterprise software costs big on license, support, hardware upgrade etc. Where as cloud based software works mostly on your browser on pay as you go basis
- Back-up and recovery: Almost you need not worry about back-up, recovery of your work. Cloud service is supposed to store it in different locations
- Easy access of information: As I mentioned, email sent with one line of soft link is much better than literally transporting the whole file through email. This makes sharing easy
- Technology infrastructure: Firms can just worry about their core business instead of setting up unnecessary expensive IT infrastructure to support their business
- Globalize in seconds: Easily deploy your business apps across globe with minimal infrastructure in place

Still, there are many more elaborative advantages of cloud computing. That is the reason, this technology is expanding fastly.

Design Trade-off

There are differences in developing traditional application and SaaS (Software As A Service) web application. Traditional application loads from consumer's hard disk to RAM. Where as, SaaS does most of the heavy lifting at server side. A thin client, typically, web browser just displays the GUI part of the application

- **Secure (SSL/TLS)** connection to your users is important. This is a little extra step. At times, this means you need to get a security key from valid CA, may need static IP, probably go through extra verification process. But quite worth. Compare the tradeoff - Your user's secure privacy vs unnecessary eavesdropping.

- **Secure development:** Many CEOs think, security is bought by getting SSL/TLS, case closed. But a hacker can get access to your system resources through secure channel as well. A hacker can employ techniques such as Buffer overrun, SQL injection etc. The trade off here is how much time/money to spend on secure development vs feature development
- **Hardware redundency Vs software redundency:** As you know, the chief reason behind cloud apps is it's reliablity. A cloud service need to have multiple/multisite backups of user files anticipating disaster. Redundency can be achieved through hardware or software. For example, take an example of investing in hardware RAID controller to installing ZFS based software redundency. This trade off is pretty tight as we have positives and negatives on both sides
- **Unlimited everything:** Be careful when you give unlimited anything. Today, it's marketing technique that we are getting used to unlimited storage, unlimited free web apps etc. This will work in short term to capture market. But from design perspective anything unlimited is not long-term sustainable. That is why many smart businesses are moving to give unlimited free for personal use,but charge for enterprise use. Let's say, you provide a spread sheet cloud service for your users. Unlimited cells, unlimited storage and unlimited everything. An user with bad intentions can bring down your server by filling in cell using some automated program. This will affect your good users who use the service genuinely. This forces you to put some cap on your users, say, 10 sheets per day, 50 MB per day etc. So the trade-off here is the the size of cap you would like to impose on your user

Brief explanation

This article is about how to write your own core cloud service. As such, building your own cloud service is an exhaustive job. It requires specialized enterprise server class hardware/software. These are standard items that can be built or bought from external vendor - capital investment. However, the key part in a cloud service is the core service that you expose to your customer through world wide web (internet). For example, google sheets expose spread sheets that can be used to create, edit, save and share with anyone easily. This such core service is the scope of this article. In this article, we will create a service in C# .net to expose spread sheet functionality with the ability to Open, edit, compute math formula, save and share a shreadsheet workbook. i.e., Spread sheet work book software as a service (SaaS)

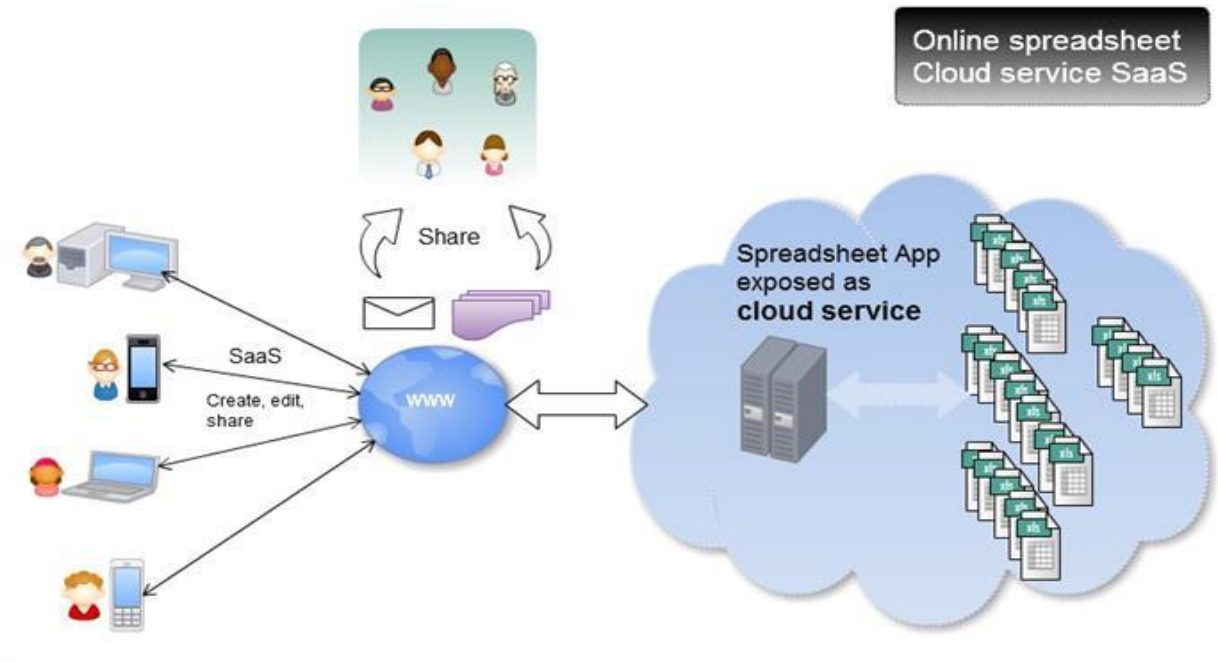


Fig 5: Cloud SaaS service

Detailed explanation

Let's begin by defining few requirements. We would like to build a service that exposes spread sheet workbook. For brevity, let's support few basic functionalities.

- User shall open workbook and get a workbook like screen
- User shall create a new work book, edit it and save it
- User shall perform math computations with the workbook

Html5 websocket is one of the major change in history of world wide web. It enables low latency, full duplex, persistent communication between web browser and your service. You can read more about this here (Reference 3). Now the key problem: low latency tcp communication between web browser and server, is solved. In case, if you need more details on how this works, please go through Reference (3) link. This project uses the html5 websocket streamer from Reference (3) link

The server side code for this spread sheet SaaS is surprisingly simple. Let's see first the main function. Main function is the entry point of a server application in C#

```
static void Main(string[] args)
 {
    html5Stream stream = new html5Stream();    // Line 1
```

```
    stream.startServer();          // Line 2
 }
```

First line of main function creates a html5Stream object. Line, two starts the service. Line 2 is a blocking call. So the server will keep running until this call returns.

Let's take a look at key part in startServer method. All this method does is accepts a new connection from the client. In our case, the client is web browser. Accept call on Socket will return when a client attempts a new connection. Upon new connection initiation, the call handling is directed to OnAcceptConnection Method. Then, it loops back to wait for another new client. This function behaves like an office reception staff. Upon a new visitor entering the work place, the reception staff redirect the guest handling to appropirate personnel and goes back to take new calls/ visitors.

```
public void startServer(string endPt = "", bool demo = false)
{
--------
--------
  while (!Console.KeyAvailable)
  {
    Socket ws = serverSocket.Accept();
    ThreadPool.QueueUserWorkItem(new WaitCallback(OnAcceptConnection), ws);

    counter++;
  }

     -------
-------
}
```

OnAcceptConnection creates uniqueId for the connected client. Going forward, this unique Id will be used to identify the client. Identifing the client using unique id helps us to present

appropriate response. After all, each client will be dealing with their own spreadsheet workbook. We need to send correct workbook data to each and everyone of them. This method checks if a file is already saved and can be opened. If file data available, then it streams the file content to the client. Otherwise, this method streams a generic initData. initData contains an empty spreadsheet workbook

```
private void OnAcceptConnection(object param)
{

   ------------
   ------------
     int rcvBytes = client.Receive(buffer);

   ----------
     clientResp = File.ReadAllText(clientInitialData[1]);
     if (clientResp.Length <= 0)
     {
        clientResp = initData;
     }

   -----------
       uniqueId = getUniqueId();
   -----------
       clientList.Add(uniqueId, clientData);

   -----------
       setStreamData(uniqueId, clientResp);
}
```

At this point, the client web browser sees an empty shreadsheet workbook. User can key in data, compute math formula using this work sheet. Whenever user completes keying in a cell and moves to next cell, an update call goes to server. Server side computes required formula, if any, then posts back the sheet to client. User do not see a page refresh. All this happens behind the scenes. Users sees the sheet updating new computations as and when she/he is typing. When user completes their work, this sheet can be saved by clicking save button. Upon save request, a special 's' is sent as first byte data. At server side, this performs a save operation. In a professional version, this save need to happen on a high performing database. Here, we are saving in file for brevity. Here is the server side code that just does that

```
private void checkSaveOrOpen(long uniqueId, ref string data)
{
   string[] dataRcvd = data.Split(new char[] {','}, 3);
   if (string.Compare(dataRcvd[0], "s") == 0)
   {
      try
      {
         data = 'u' + data.Remove(0, 1);
         File.WriteAllText(dataRcvd[1], data);
      }
      catch (Exception ex)
      {
         System.Console.WriteLine("Error: Save failed for clientId: {0}, {1}", uniqueId, ex.Message);
      }
   }
   ----------
----------

}
```

The above code checks if letter 's' is present in the first byte of data received. If yes, then it flips the byte and saves the data to a file. It flips the byte to 'u' to make sure save is complete. This helps avoid repeatedly saving for every client request. Lastly, spreadsheet computations are performed by muParser. This is a fast math expression parser. Please read the Article in Reference (3), which uses this parser library

This completes one round trip of features presented by this spreadsheet.

Points of Interest

- All functionalities of professional spreadsheet can be added to this project. For simplicity this online spread sheet comes with basic functionalities such as create new sheet, Open existing sheet, Save sheet and Math formula computation. The aim of this article/project is to explain how to build a basic working prototype. A professional worksheet needs work. Contact me in case if you need a professional one
- There is a little functionality that shows the Spreadsheet is online or offline. A small red light blinks at the top if the sheet is offline. A green light represents the sheet is online. This little functionality is achived by just two lines of code. These two methods in client side with a gif file gives this nice little helpful feature

```
function statusDisconnect()
{
    statusLED.src = "redlight.gif"
    statusTxt.innerHTML  = "Not Connected";
}


function statusConnected()
{
    statusLED.src = "greenlight.gif"
    statusTxt.innerHTML  = "Connected";
}
```

- At client side, handling keyboard input such as backspace, arrow keys req, regular keys need extra code. For simplicity and seamless working I used an html input box to act as

our input. Whatever user types in html input box is transfered to the shpreadsheet cell. This way, the client side code to handle keyboard is avoided

- If you notice the computed columns are not working as expected then check if you have muParser.dll in your service's working directory

My sincere thanks to..

I would like to thank you for reading this article. I would love to hear back from you. If you like this article, please vote for this article at top of this page and leave your comment.

Many thanks to muParser, the math parsing library used in this grid (Reference 4). It's pretty fast expression parse library. Thanks to https://cacoo.com. I used their online tool for creating diagrams for this article.

References

1. Software as a service
2. Google docs
3. Html websocket stream
4. Math expression parser